

Servicios mal configurados

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Los servicios de Windows suelen correr como SYSTEM. Si tu usuario puede modificar la configuración del servicio (binPath), reemplazar su ejecutable, o si la ruta del binario no está entre comillas y hay un directorio escribible en medio, puedes hacer que el servicio ejecute TU binario como SYSTEM. Es el winprivesc-modelo aplicado a servicios.

Instaladores descuidados dejan servicios con permisos NTFS flojos en el .exe o permiten a usuarios normales reconfigurar el servicio. Y una ruta como C:\Program Files\My App\svc.exe sin comillas hace que Windows pruebe C:\Program.exe primero: si puedes escribir ahí, secuestras el arranque.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). WinPEAS marca servicios modificables, binarios escribibles y unquoted paths.

Cómo identificarla

- accesschk o WinPEAS marca SERVICE_CHANGE_CONFIG o WRITE sobre un servicio.
- El .exe del servicio está en una carpeta escribible por tu usuario.
- Una ruta de servicio con espacios y sin comillas, con un dir intermedio escribible.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

sc qc <servicio> para ver BINARY_PATH_NAME y SERVICE_START_NAME; cruzalo con accesschk.

Evidencia esperada: Una ruta sin comillas con directorio intermedio escribible, o permiso de reconfigurar el servicio (SERVICE_CHANGE_CONFIG).

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sc qc <servicio>
```

Objetivo y contexto: Muestra la configuración del servicio: su BINARY_PATH_NAME y la cuenta con la que corre. Ahí ves si la ruta está sin comillas y si corre como LocalSystem.

Resultado esperado: BINARY_PATH_NAME (mira comillas y espacios) y SERVICE_START_NAME (LocalSystem = SYSTEM). Combinar con accesschk revela si puedes modificarlo.

Capa 2: prueba documentada

```
sc config <servicio> binPath= "C:\Windows\Temp\pe.exe" && sc start <servicio>
```

Objetivo y contexto: Si tienes SERVICE_CHANGE_CONFIG, reescribes que ejecuta el servicio y lo arrancas: tu binario corra como SYSTEM. Es el abuso más directo.

Resultado esperado: El servicio ejecuta tu pe.exe como SYSTEM (crea un usuario admin o lanza tu shell). Si no puedes cambiar config, prueba reemplazar el .exe o el unquoted path.

Anatomía de los payloads

La primera prueba es `sc qc <servicio>`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una ruta sin comillas con directorio intermedio escribible, o permiso de reconfigurar el servicio (SERVICE_CHANGE_CONFIG).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera servicios y sus permisos (accesschk / sc qc).
- Según el fallo: reconfigura binPath, reemplaza el .exe, o coloca un exe en la ruta sin comillas.
- Reinicia el servicio (o espera) para que ejecute tu payload como SYSTEM.
- Confirma con whoami que eres nt authority\system.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Todos los servicios corren con permisos correctos y rutas entre comillas.
- No tienes SERVICE_CHANGE_CONFIG ni escritura sobre ningún binario de servicio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un servicio que corre como SYSTEM con permisos flojos: si modificas su binario, su binPath o una ruta sin comillas, ejecutas como SYSTEM. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (service-misconfig-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.